THE FRONTEND PROJECT PLAYBOOK

A practical project roadmap after HTML, CSS and JavaScript theory

YOUR GOAL

Turn your HTML, CSS and JavaScript knowledge into working projects. Build the projects in order, learn by solving real problems, and finish with a small portfolio you can explain confidently.

What is inside

- Six projects ordered from beginner to harder frontend applications.
- Feature lists and the JavaScript / frontend concepts each project practises.
- Clickable links for design inspiration, images, icons, APIs and deployment.
- A repeatable project workflow and a checklist for the first project.

Suggested pace: about 4–6 weeks for the first four small projects, then continue at a pace that lets you understand and debug your code.

CORE RULE Do not copy a complete tutorial. Plan the feature, attempt it yourself, search for specific blockers, and write the code yourself.

Project sequence at a glance

Complete one project before starting the next. These durations are estimates, not deadlines; spend extra time where you are still unsure.

#	PROJECT	LEVEL	MAIN LEARNING
1	Personal Portfolio	Easy	Responsive layout, DOM, forms
2	Expense Tracker	Easy–Medium	Arrays, DOM, localStorage
3	E-commerce Product Store	Medium	Search, filters, cart state
4	Weather Application	Medium	fetch, async/await, APIs
5	Task Management App	Medium–Hard	Events, state, drag-and-drop
6	Learning Platform Frontend	Hard	Multi-page structure, reusable UI

Before you begin

- Use plain HTML, CSS and JavaScript for Projects 1–5. This will strengthen your fundamentals before you move to a framework.
- Make each project responsive: test it on both desktop and a mobile-sized screen.
- Keep your code in GitHub. Make small commits when a feature works, rather than uploading only at the end.
- The UI images shown in design galleries are inspiration, not a requirement to reproduce them pixel-for-pixel. Build your own version.

> **A GOOD FINISH LINE**
> A project is complete when its main features work, the interface is usable on mobile, refreshing does not unexpectedly lose saved data (where persistence is required), and you can explain your code.

PROJECT 01 / Personal Portfolio Website

EASY | Suggested time: 3–4 days

Build a website that introduces you and presents your skills, projects and contact information. It can later become the home page for your developer portfolio.

What to build

- Home / hero section with your name, role and a clear short introduction.
- About Me section with a few lines about what you are learning.
- Skills section with technologies you know.
- Projects gallery with project name, short description and links.
- Contact section with a form and clear contact links.
- Responsive mobile navigation menu.

Concepts to practise

- CSS Flexbox and Grid, spacing, typography, responsive breakpoints and hover states.
- DOM selection, event listeners, form validation and toggling the mobile menu.

Images and design resources

USE	RESOURCE	WHY / NOTES
Photos	Unsplash	Free-to-browse photo library; choose an appropriate profile or background image and check the image licence/terms.
Icons	Lucide Icons	Simple, consistent icons for GitHub, email, menus and project actions.
Fonts	Google Fonts	Choose one readable font family and use consistent weights.
Design	Figma	Optional: sketch a rough layout before coding.

Extra challenge

- Add a dark-mode toggle, filter buttons for projects, and a “back to top” button.

DONE WHEN

Every section is reachable, the layout works on mobile, the contact form validates inputs, and all project links work.

PROJECT 02 / Expense Tracker

EASY–MEDIUM | Suggested time: 4–6 days

Build a working expense manager that lets a user record spending and understand where money is going. This is an excellent project for practising JavaScript state and data updates.

What to build

- Add an expense with a title, amount, category and date.
- Display all transactions in a list or table.
- Edit and delete transactions.
- Calculate total spending and show a monthly total.
- Search or filter transactions by category or date.
- Save data in localStorage so it remains after a refresh.

Concepts to practise

- Arrays and objects; array methods such as map(), filter() and reduce().
- DOM manipulation, form validation, dates, number formatting and localStorage.

Resources

USE	RESOURCE	WHY / NOTES
Charts	Chart.js	Optional library for category or monthly spending charts. Start with totals before adding charts.
Icons	Lucide Icons	Optional icons for edit, delete, search and category labels.

External photos are not required. Use CSS, simple icons and charts rather than spending time searching for pictures.

Extra challenge

- Add a monthly budget and show a warning when spending reaches the limit.

DONE WHEN

You can create, edit, delete, search and reload saved expenses without losing the data. Check invalid amounts and empty form fields.

PROJECT 03 / E-commerce Product Store

MEDIUM | Suggested time: 5–7 days

Build a shopping frontend that presents products and allows a user to search, filter, sort and manage a shopping cart. This is a frontend demonstration; real payments and secure accounts need a backend and payment provider.

What to build

- Product cards containing an image, name, price and category.
- Search products by name.
- Filter by category and sort by price.
- Add items to a cart and remove them.
- Change quantity and calculate the cart total.
- Save the cart in localStorage.
- Optional product details page and separate cart page.

Concepts to practise

- map(), filter() and reduce(); dynamic rendering from an array of product objects.
- Event listeners, cart state, quantity calculations, conditional UI and localStorage.

Product data and image sources

USE	RESOURCE	WHY / NOTES
Sample data	Fake Store API	A practice API that returns sample product data, including image URLs. Availability and fields may change; inspect the returned JSON.
Photos	Unsplash	Use if you want your own product photography instead of sample API images.
Icons	Lucide Icons	Cart, search, filter, plus/minus and close icons.

Extra challenge

- Create a product details page and a “sort by price: low to high” option.

> **IMPORTANT**
> Do not collect card details or pretend a demo cart is a secure checkout. Keep this version as a frontend learning project.

PROJECT 04 / Weather Application

MEDIUM | Suggested time: 3–5 days

Build an application that requests real weather information for a city or location and handles the states between starting a request and showing the result.

What to build

- Search for a city or location.
- Show current temperature and weather conditions.
- Show humidity and wind speed.
- If supported by the selected endpoint, show a multi-day forecast.
- Display a loading message while data is being fetched.
- Show a helpful message for invalid locations or network/API errors.
- Optional recent-search list.

Concepts to practise

- fetch(), promises, async/await, JSON, API endpoints and response handling.
- Loading, success and error states; guarding against missing or unexpected data.

Data and visual resources

USE	RESOURCE	WHY / NOTES
Weather data	Open-Meteo	Weather API. The standard service is available without an API key for non-commercial use under its applicable terms; review its current usage rules before publishing or commercial use.
Weather icons	OpenWeather Conditions	Weather-condition and icon reference. Check its current terms/licensing before reusing the assets.

Extra challenge

- Remember recent searches and allow users to switch between Celsius and Fahrenheit.

DONE WHEN

The app shows live data, handles loading and errors, and does not crash when a request fails or a field is missing.

PROJECT 05 / Task Management App

MEDIUM–HARD | Suggested time: 6–8 days

Build a small Trello-style board with tasks grouped by status. This project adds more complex state changes and interactions.

What to build

- Create, edit and delete tasks.
- Use To Do, In Progress and Completed columns.
- Add task priority and due date.
- Search tasks by title.
- Drag and drop a task between columns.
- Persist tasks in localStorage.
- Optional statistics showing the number of tasks in each status.

Concepts to practise

- DOM events, event delegation, application state, rendering updates and drag-and-drop events.
- Separating data logic from UI logic so a change can be tested and debugged more easily.

Resources

USE	RESOURCE	WHY / NOTES
Icons	Lucide Icons	Use simple icons for add, edit, delete, status and search controls. No photos are required.

Extra challenge

- Add a deadline filter such as “due today” and a confirmation before deleting a task.

DONE WHEN

A task can be created, edited, deleted, searched, moved between statuses and restored after refreshing the browser.

PROJECT 06 / Online Learning Platform Frontend

HARD | Suggested time: 7–10 days

Build a small student learning portal with course cards, lesson navigation and progress indicators. Build this in plain JavaScript first; later, rebuild it with Vue to learn components and routing.

What to build

- Course listing page with search.
- Course detail page showing course information and lessons.
- Lesson navigation (next / previous).
- Student dashboard with enrolled courses.
- Progress indicators for lessons completed.
- Save enrolled courses or progress locally.
- Responsive layouts for mobile and desktop.

Concepts to practise

- Reusable UI patterns, dynamic lists, state changes, multi-page structure and routing concepts.
- Data modelling for courses, lessons and progress. Keep content data separate from the page structure.

Image and design resources

USE	RESOURCE	WHY / NOTES
Photos	Pexels	Search for education, students or workspace images; check licence/terms for your intended use.
Photos	Unsplash	Alternative source for course thumbnails and general learning imagery.
Design	Figma	Optional: plan the dashboard and course page layout before coding.
Icons	Lucide Icons	Course, progress, play, menu and profile icons.

Extra challenge

- After completing the plain JavaScript version, rebuild it in Vue.js and connect it to a real backend API.

DONE WHEN

Users can browse courses, open a course, move through lessons and see progress saved locally. Make it clear that the version has demo/local data, not real user accounts.

Repeat this workflow for every project

1. Plan the result

Write down the features. Sketch the page on paper or plan it in Figma. Decide what the user can do and what should happen after each action.

2. Build the HTML structure

Create meaningful sections, headings, forms, buttons and lists. Give controls clear labels and use appropriate HTML elements.

3. Style with CSS

Build the layout, typography, spacing, responsive behaviour and visible focus/hover states. Test a narrow screen early.

4. Add JavaScript one feature at a time

Start with one working interaction. Use small functions and descriptive names. Avoid adding all features before testing any of them.

5. Test and debug

Try empty inputs, incorrect values, repeated clicks, refreshes, slow/offline network and unexpected API responses. Check the browser console.

6. Publish and document

Push the project to GitHub. Add a README describing the goal, features, technologies, how to run it and known limitations. Deploy it if suitable.

Useful publishing links

Code repository: GitHub

Static site hosting: GitHub Pages

Design planning: Figma

Start today: Expense Tracker checklist

Keep Version 1 deliberately small. Complete these five steps in order. Add charts and extra features only after the core workflow works.

☐	**1. Create the HTML layout and expense form** Include fields for expense title, amount, category and date, plus an Add button.
☐	**2. Style the page for mobile and desktop** Use a clear form, readable transaction list, spacing and responsive layout.
☐	**3. Add expenses dynamically with JavaScript** Read form values, validate them, create a transaction object and update the UI.
☐	**4. Calculate total spending** Use array methods to calculate and display the total and monthly total.
☐	**5. Save and load transactions with localStorage** Load saved data on page start; save whenever transactions change.

Definition of done

- The expense can be added, edited, deleted and searched.
- Empty or invalid amounts are rejected with a useful message.
- The total updates when an expense changes.
- Saved transactions return after refreshing the browser.
- The page is usable on a phone-sized screen.
- No unresolved errors appear in the browser console.

AFTER THIS PROJECT

Build the e-commerce store next. Then build the weather app, where you will learn to request external data and handle loading and error states.

Final rules to remember

- Do not start all six projects at once. Finish one before moving on.
- Do not switch technology stacks repeatedly. Strengthen your current JavaScript first.
- Use AI to understand an error or concept, not to generate the entire project for you.
- A working project you can explain is worth more than several copied tutorial projects.
- Keep source code, screenshots, a README and a live demo link together for each finished project.